

Progress Report

Day 24 — Reports Dashboard

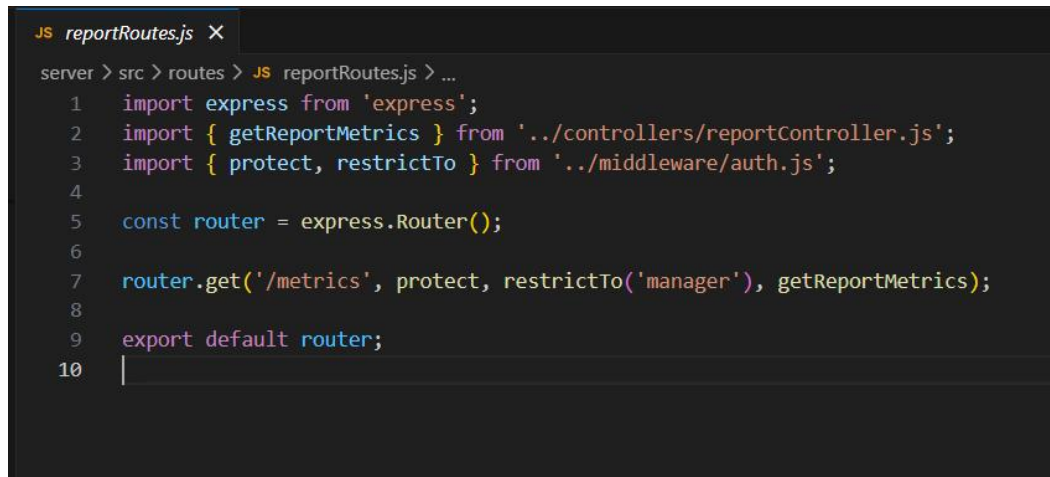
Task Management MERN Stack Platform

Shafin Nigamana | 12 June 2026

Tasks Completed

2.1 Reports Metrics API

- Created a dedicated reports endpoint for aggregated project metrics.
- Implemented manager-only access control on report routes.
- Combined audit log data and task data into a unified reporting response.



```
JS reportRoutes.js X
server > src > routes > JS reportRoutes.js > ...
1  import express from 'express';
2  import { getReportMetrics } from '../controllers/reportController.js';
3  import { protect, restrictTo } from '../middleware/auth.js';
4
5  const router = express.Router();
6
7  router.get('/metrics', protect, restrictTo('manager'), getReportMetrics);
8
9  export default router;
10 |
```

Figure 2.1: Reports metrics endpoint secured using manager-only authorization.

2.2 Tasks Closed Per Week

- Implemented weekly task completion aggregation.
- Retrieved completed task activity from audit log records.
- Generated weekly completion statistics for reporting purposes.

```

JS reportController.js X
server > src > controllers > JS reportController.js > ...
1 import { createMySQLPool } from '../config/mysql.js';
2 import User from '../models/User.js';
3 import Task from '../models/Task.js';
4 import mongoose from 'mongoose';
5
6 export const getReportMetrics = async (req, res) => {
7   try {
8     const pool = createMySQLPool();
9
10    // 1. Metric 1: Tasks Closed Per Week
11    const [closedRows] = await pool.query(`
12      SELECT
13        YEARWEEK(created_at, 1) AS week,
14        COUNT(*) AS closed_count
15      FROM audit_log
16      WHERE action = 'UPDATE_TASK'
17        AND JSON_UNQUOTE(JSON_EXTRACT(payload_json, '$.updatedFields.status')) = 'done'
18      GROUP BY YEARWEEK(created_at, 1)
19      ORDER BY week DESC
20    `);
21
22    // 2. Metric 2: Top 5 Contributors
23    const [contributorRows] = await pool.query(`
24      SELECT actor_id, COUNT(*) AS actions
25      FROM audit_log
26      WHERE actor_id != 'SYSTEM'
27      GROUP BY actor_id
28      ORDER BY actions DESC
29      LIMIT 5
30    `);
31
32    // Fetch user details for contributors from MongoDB
33    const userIds = contributorRows
34      .map((row) => row.actor_id)
35      .filter((id) => mongoose.Types.ObjectId.isValid(id));
36
37    const users = await User.find({ _id: { $in: userIds } }).select('_id name');
38    const userMap = new Map(users.map(u => [u._id.toString(), u.name]));
39

```

Figure 2.2: Weekly task completion metrics generated from audit log data.

2.3 Top Contributors

- Implemented contributor ranking based on recorded platform activity.
- Aggregated audit records by user activity count.
- Displayed the top five contributors across the platform.

Rank	Name	Actions
#1	Shafin Nigamana	31 actions
#2	Manager User	6 actions

Figure 2.3: Top contributors section displaying activity rankings.

2.4 Overdue Rate Metrics

- Calculated overdue task statistics using active task records.
- Measured overdue tasks against total active tasks.
- Generated overdue percentage metrics for management reporting.

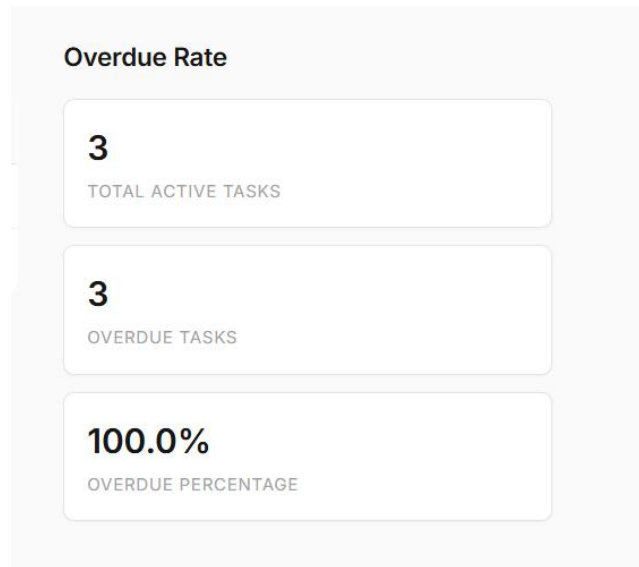


Figure 2.4: Overdue rate metrics showing active, overdue, and percentage values.

2.5 Reports Dashboard

- Created a dedicated Reports page for managers.
- Displayed project metrics in a structured dashboard layout.
- Added loading and error handling for report retrieval.

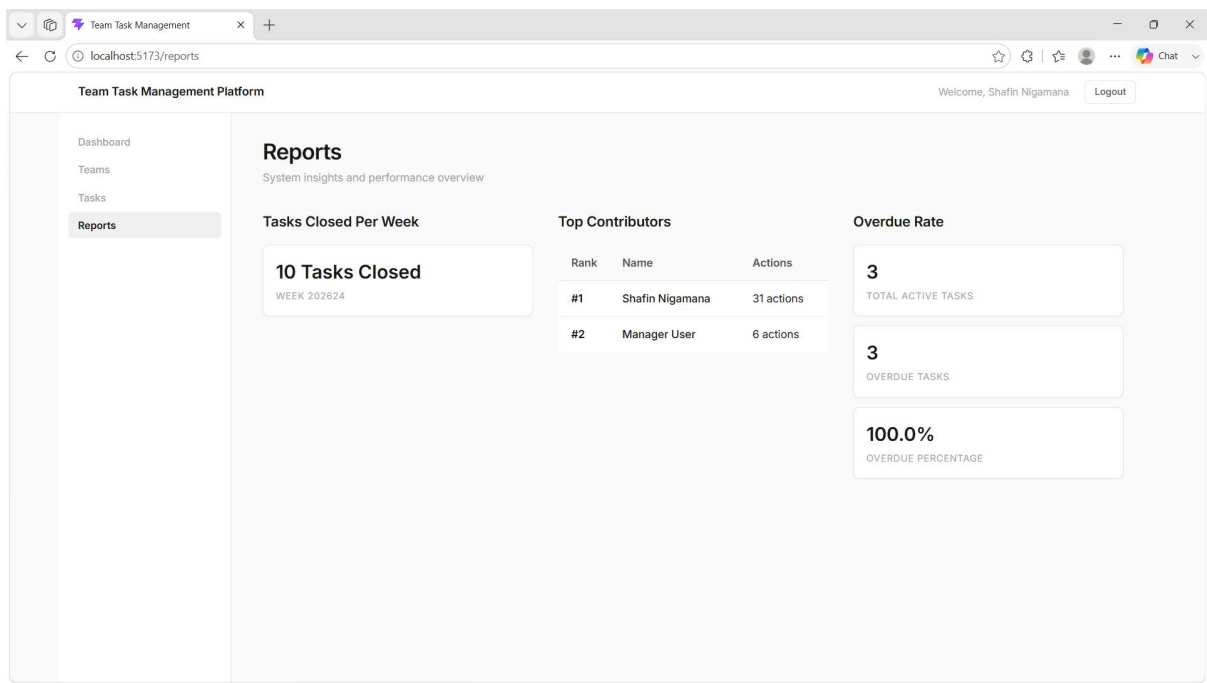


Figure 2.5: Reports dashboard displaying weekly closures, contributor rankings, and overdue statistics.

2.6 GitHub Update

Changes were committed and pushed to the project repository after successful testing and verification.

